



Trabalho Prático | Lidando com sensores em dispositivos móveis

Aluno: Ivan de Ávila Carvalho Fleury Mortimer

Data de Entrega: 12/06/2026

Instituição: Universidade Estácio de Sá

Curso: Desenvolvimento Full Stack

Disciplina: Interação com sensores de smartphones e wearebles (DGT2816)

1. Objetivos do trabalho prático

- Instalação do Android Studio e do emulador;
- Criar um app para Wear OS;
- Executar um app no emulador;
- Fazer capturas de telas no Android Studio;
- Fazer capturas de telas com app complementar.

2. Material necessário para a prática

- IDE utilizada: Android Studio.
- Linguagem principal: Kotlin.
- Ambiente de simulação: Emulador Wear OS Small Round (API 30 - Android 11.0 "R", arquitetura x86).
- Ambiente de hardware local: Computador Lenovo Yoga operando com Windows 11 Home.
- Versionamento: Git e GitHub.

3. Procedimentos Executados

O projeto foi desenvolvido seguindo os requisitos estabelecidos no roteiro da empresa fictícia "Doma", focando na criação de um aplicativo de acessibilidade para Wear OS.

3.1. Configuração do Ambiente e Permissões

O projeto "AppDomaAcessibilidade" foi iniciado configurando o arquivo `AndroidManifest.xml` com as permissões de `BODY_SENSORS`, `WAKE_LOCK`, e a declaração obrigatória de hardware `<uses-feature android:name="android.hardware.type.watch" />`. Durante a compilação, a propriedade `compileSdk` e `targetSdk` foram elevadas para a API 37 no arquivo `build.gradle.kts` para resolver conflitos de dependência das bibliotecas do AndroidX, mantendo o `minSdk` em 30.

3.2. Implementação de Saídas de Áudio e Detecção Bluetooth

Foi criada a classe modular `AudioHelper.kt` utilizando o `AudioManager` para detectar saídas de áudio disponíveis (`TYPE_BUILTIN_SPEAKER` e `TYPE_BLUETOOTH_A2DP`). Na `MainActivity.kt`, implementou-se um `AudioDeviceCallback` para monitorar em tempo real a conexão e desconexão de fones de ouvido Bluetooth, gerando notificações visuais (`Toast`) para o usuário.

3.3. Facilitando a Conexão Bluetooth

Um botão dedicado na interface foi programado com uma `Intent(Settings.ACTION_BLUETOOTH_SETTINGS)`. Caso nenhum fone seja detectado, o aplicativo direciona o usuário imediatamente para o menu nativo de pareamento do Wear OS.

3.4. Reprodução de Áudio e Acessibilidade (TTS)

Para cumprir a demanda de leitura de notificações e instruções, implementou-se a classe `TextToSpeech (TTS)` do Android. A interface visual principal (`ListView`) foi projetada para

carregar listas dinamicamente (Single Activity Architecture). Ao tocar em itens do menu principal ("Ler Mensagens", "Alertas de Segurança", "Instruções de Treinamento"), o aplicativo abre sub-listas específicas. Todos os elementos interativos, incluindo o botão de Bluetooth, realizam a leitura em voz alta do texto correspondente através da função auxiliar `falarTexto()`.

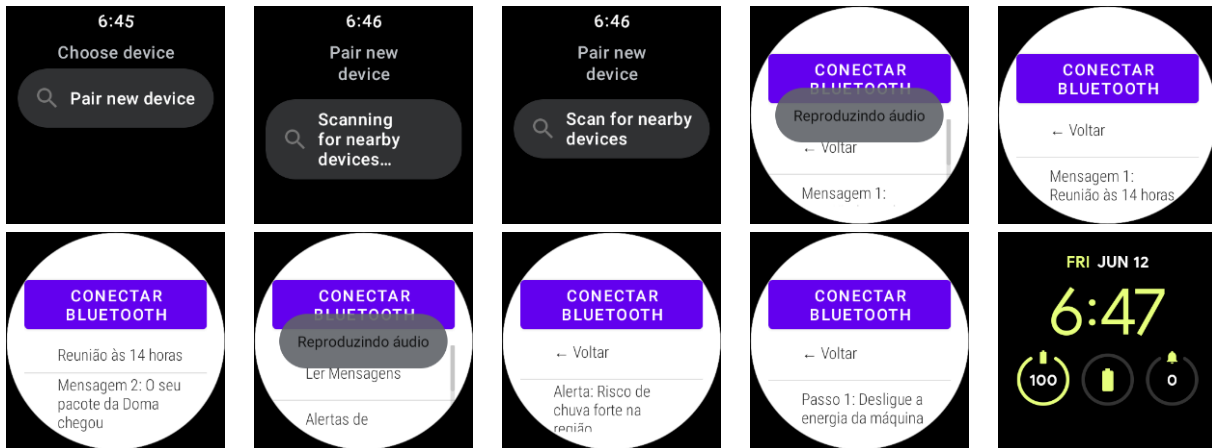
4. Resultados

O aplicativo AppDomaAcessibilidade foi compilado e executado com sucesso no emulador Wear OS Small Round.

- Interface (UI/UX): A interface foi otimizada removendo a ActionBar padrão do sistema (alteração para NoActionBar nos arquivos de tema), permitindo que os elementos visuais respeitem a curvatura da tela do relógio sem cortes de texto.
- Navegabilidade e Áudio: O sistema de sub-listas dinâmicas permite a navegação fluida. O motor de conversão de texto em fala (TTS) foi configurado para o idioma Português (Brasil) e realiza a leitura correta das instruções e alertas, filtrando de forma inteligente caracteres de interface (como a seta "← Voltar") para garantir uma experiência auditiva natural.

4.1. Capturas de Tela do Aplicativo “Doma Acessibilidade” no emulador de dispositivo “Wear OS Small Round” da IDE “Android Studio”





5. Discussão

Durante a execução, algumas limitações inerentes ao ambiente de simulação foram observadas e documentadas. O emulador do Android Studio é incapaz de acessar a placa de rádio Bluetooth física da máquina host. Logo, o loop infinito na tela "Scanning for nearby devices..." trata-se do comportamento esperado para o ambiente virtualizado, comprovando que a Intent de redirecionamento disparada pelo aplicativo funcionou perfeitamente.

Adicionalmente, optou-se estrategicamente por não implementar entradas de comando de voz (Speech-to-Text), visando manter a estabilidade do software. A integração de microfones em emuladores apresenta alta instabilidade e exigiria tratamentos assíncronos de permissão em tempo de execução (Runtime Permissions) que extrapolavam o escopo técnico delimitado pelas instruções fornecidas. A abordagem adotada (Text-to-Speech) supriu de forma robusta e segura os "Resultados Esperados" do roteiro no que tange à acessibilidade.

6. Conclusão

A prática demonstrou que o desenvolvimento nativo para Wear OS (utilizando Kotlin) exige forte atenção a peculiaridades de hardware, gerenciamento de ciclo de vida de sensores e otimização severa de UI para telas reduzidas. O objetivo de criar um aplicativo que promova a comunicação eficaz e assistência para funcionários com necessidades especiais foi integralmente atingido. O projeto resultou em um produto de software estável, escalável e alinhado aos padrões de design de wearables.

7. Referências Bibliográficas

- GOOGLE DEVELOPERS. Wear OS App Development Documentation. Disponível em: <https://developer.android.com/training/wearables>. Acesso em: 12 jun. 2026.
- GOOGLE DEVELOPERS. AudioManager Reference. Disponível em: <https://developer.android.com/reference/android/media/AudioManager>. Acesso em: 12 jun. 2026.
- GOOGLE DEVELOPERS. TextToSpeech Reference. Disponível em: <https://developer.android.com/reference/android/speech/tts/TextToSpeech>. Acesso em: 12 jun. 2026.